

Jason Ablott

12/05/2025

CS 6019: MSD Capstone

CAPE: The Most Accurate Weather App

Introduction and Background

CAPE is a cross platform (Android and iOS compatible) mobile application designed to be simple and easy to use. The main problem this application solves is the lack of a mobile application offering from the National Weather Service. The National Weather Service is a U.S. government agency whose focus is on weather data collection and forecasting. Because of their extensive resource availability, they are well known as the most accurate source of weather observational data and forecasting. Unfortunately, this knowledge is niche to weather enthusiasts and the general populace still obtains this information from private companies who collect user data, have annoying advertisements embedded in their products, and also offer much less reliable data and forecasts. These issues can all be solved or improved by utilizing the free public services that the National Weather Service offers, but there is a major barrier in that their data is only accessible via their website, weather.gov. This limits the reach of the quality information, and many users instead rely on a mobile application such as Apple Weather for this information. These products meet basic user needs, but they have the previously mentioned limitations and are especially lacking in accuracy.

CAPE solves this issue and fills the void of a mobile application that uses the same data and forecast information that is available from weather.gov. The NWS offers a free API to allow

access to all this data, and CAPE utilizes many API endpoints to fetch data and display it in a simple navigable format for anyone to use.

There are several other features that CAPE offers to improve usability over existing products as well. In addition to being much more accurate with forecasting due to utilization of NWS data, CAPE also allows persistent storage of forecast information for offline use. Current applications and weather.gov rely on an active network connection to fetch data and cannot be used in absence of that. CAPE caches data locally to allow users to close and re-open the application and still have access to the previously fetched forecast information. This solves a personal problem of mine where I would have to take screenshots of weather.gov forecasts before traveling to areas with no cell reception for camping, hiking, or other outdoor activities. This is one of the most practical improvements this product provides over existing products available to the general public.

CAPE also integrates one of the most useful features that weather.gov offers, but mobile application offerings lack, point forecasting. Point forecasting allows the user to input any location (within the United States) and fetch an accurate forecast for that exact location. This is only available from the National Weather Service due to the forecast grids their local forecast offices create. This feature is incredibly useful to anyone accessing outdoor recreation, especially in mountainous terrain as the forecast grids accurately account for elevation among many other location specific properties to generate an accurate forecast for any location. CAPE offers this ability to use a fully interactive map for the user to select any location for a point forecast. This feature is a major benefit for a large amount of users who would previously have to be familiar with and use weather.gov for this data, or go without it entirely.

While designing CAPE, the goal was to keep the product simple and easy to use. There is no need to make weather forecasts complicated or add unnecessary features, we just want to get the information we need in a readable and understandable format. This led to a 4-screen solution where each screen offers one core functionality.

Screens/UI Structure

- **Home Screen/Current Conditions:** This screen displays current weather information for the selected location.
- **7 Day Forecast Screen:** This screen displays a 7-day forecast for the selected location.
- **Hourly Forecast Screen:** This screen displays an hourly forecast for the selected location.
- **Location Selection Screen:** This screen allows the user to search for any location via a text field, via a fully interactive map, or allow the app to use their current location to fetch relevant forecast and weather data on the other screens.

This visual layout with universal tab navigation at the bottom of the screens makes the application simple and user friendly, while displaying only the most useful information without anything extra. CAPE was designed using React Native as the foundation for the components to be able to offer this as a cross-platform product and provide access to valuable data to users of both Android and iOS.

Solution Description

CAPE is a cross-platform mobile application providing a simple 4-screen interface for important weather and forecast information. It is targeted towards users of mobile weather applications, and at weather and outdoor enthusiasts who utilize the National Weather Service as a source of weather information via their only public interface, weather.gov.

Components:

The application design is divided into 3 main components. The code is separated into a user interface layer, a state and data management layer, and an interface layer to interact with APIs. The UI components are organized using expo-router with a tab-based layout. Within each individual screen reusable UI components are used where applicable. The screens are the Home, 7-day, Hourly, and Location screens. See results section below for more details on each screen. The state layer is simple, and tracks saved location information and cached data, while the interface layer contains all functions that interact with external APIs to fetch data from various sources listed below in the APIs subsection. See figure 1 below for a visual representation of the high-level components of the application.

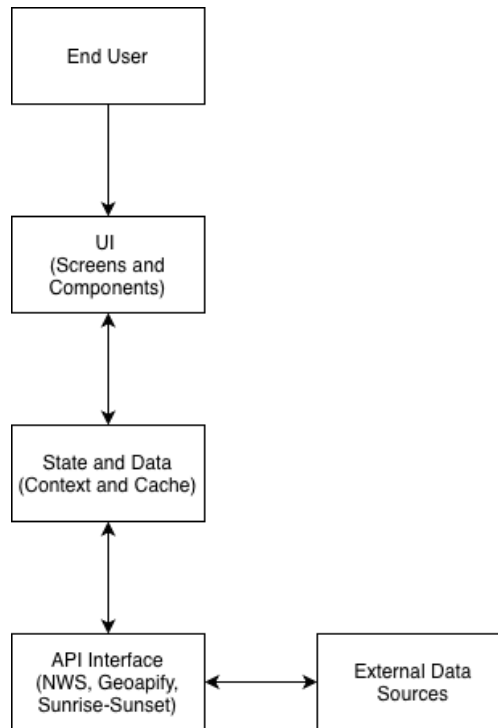


Figure 1: Main structure and high-level components

Tools:

React Native is used as the foundation of the application. This is implemented via Expo, which provides a development server, device APIs, and many other very useful features to make the development experience much simpler. Navigation is handled by expo router, a built-in file-based routing system. The base level code is written in TypeScript in the react framework and executed using npm.

Tools used:

- **Android Emulator** (Via AndroidStudio): Used for testing.
- **iOS Simulator** (via Xcode): Used for testing.
- **Expo Go**: Used for real on-device iOS testing and user feedback collection.

- **AsyncStorage**: Used to implement cached weather data for offline functionality.
- **Jest and React Native testing-library**: Used for unit testing.
- **VSCode**: Used for development and project structuring.

APIs used:

- **National Weather Service**: Endpoints to get gridpoint, 7-day forecast, hourly forecast, location info, and active alerts were used.
- **Sunrise-Sunset.org**: Endpoint to get sunrise and sunset times was used.
- **Geoapify**: Endpoint to autofill location names was used.

Software organization/design

The overall design follows a layered structure to separate UI, state management, and data access. The UI layer handles layout, rendering, and navigation. Business logic and caching are handled in the state management layer. All external communication with APIs is within the dedicated API interface layer. This structure keeps the application modular and testable.

Major Modules:

- **UI Screens and re-usable components** (building up the UI efficiently)
- **Location context and cache management** (storing necessary data and state)
- **API service modules** (grid lookup, response parsing, etc.)

The UI layer is built on top of the expo-router file system, which allows for tab-based navigation with individual route files for each of the screens. Each screen/view uses re-usable components

for visual elements including cards, loading indicators, headers, and icons. This allows for consistent styling and prevents repeated code.

The state management and data layer manages application wide state and caching. React Context is used for storing location data so all screens can access one single source of truth. Weather data is cached using AsyncStorage along with timestamps, allowing the app to load cached data to improve performance and support offline use.

The API layer contains focused functions that are responsible for calling the API endpoints listed above. This layer handles URL construction, response validation and processing, and mapping raw API JSON output into usable formats. The isolation of this layer allows for the easy adaptation to API changes and ensures easier testing.

Algorithm Overview:

CAPE operates by first attempting to read cached forecast data. If cached data is available, it is displayed immediately while updated data is fetched in the background. If updated data is successfully fetched, it is parsed and used to update the cache. The high-level logic of the application is relatively simple and relies on many small functions/components to complete this high-level task. The finer details of that functionality are beyond the scope of this report and can be seen in detail in the source code.

Timeline and Challenges

The timeline for working on this project split work into 3 main sections: research, implementation, and testing/debugging. The week-to-week work flowed in the proposed order; however, time allocation was less than ideal during the first 11 weeks of the project due to other time commitments outside of the project. This unfortunately did impact the final product in

respect to the number of features that are fully implemented at this stage of development. The choice of React Native (a framework I have very limited background in) and TypeScript (I had no experience with, only some JavaScript experience) did lead to more time needing to be dedicated to learning the framework and syntax than might have been ideal. This also impacted the amount of productive time that could be spent on writing code and implementing functionality. The biggest lesson learned from a planning and implementation perspective is that more architecture planning and research early on would have helped the most with the final project result. While the implementation would look pretty much the same, the code is not organized as well as it should be, and some things are not structured in the ideal way. This can be hard to know how to do at the beginning of a project, but more time should have been dedicated to this part of the project before jumping into coding and implementation.

Results

The resulting product is a simple, user-friendly mobile application that is fully functional on both Android and iOS devices. The final product contains over 3200 lines of user written code (including comments and a rough testing suite containing 10 test suites and 44 total tests), and approximately 26 user generated files. The entire project is much larger due to the Expo files that handle many other tasks such as routing, UI elements, etc.

The home screen successfully loads active weather watches and warnings issued by the National Weather Service, allowing the user to select and read the full text of each. It also fetches the sunrise and sunset times for the location from another free API that provides that data for any location. This is an add on feature that is also not available from the National Weather Service but was added as a valuable tool at the request of users when testing prototypes of the

application. It then displays current conditions including temperature, wind speed, and humidity. This also shows when the conditions were observed and displays an icon for visual representation of the observed conditions. Finally, the home screen shows the high and low forecast temperatures for the day, and the current forecast for the next 12-hour period (either today or tonight) for quick reference. See below for a screenshot of the home screen on both an iOS and Android emulated device.

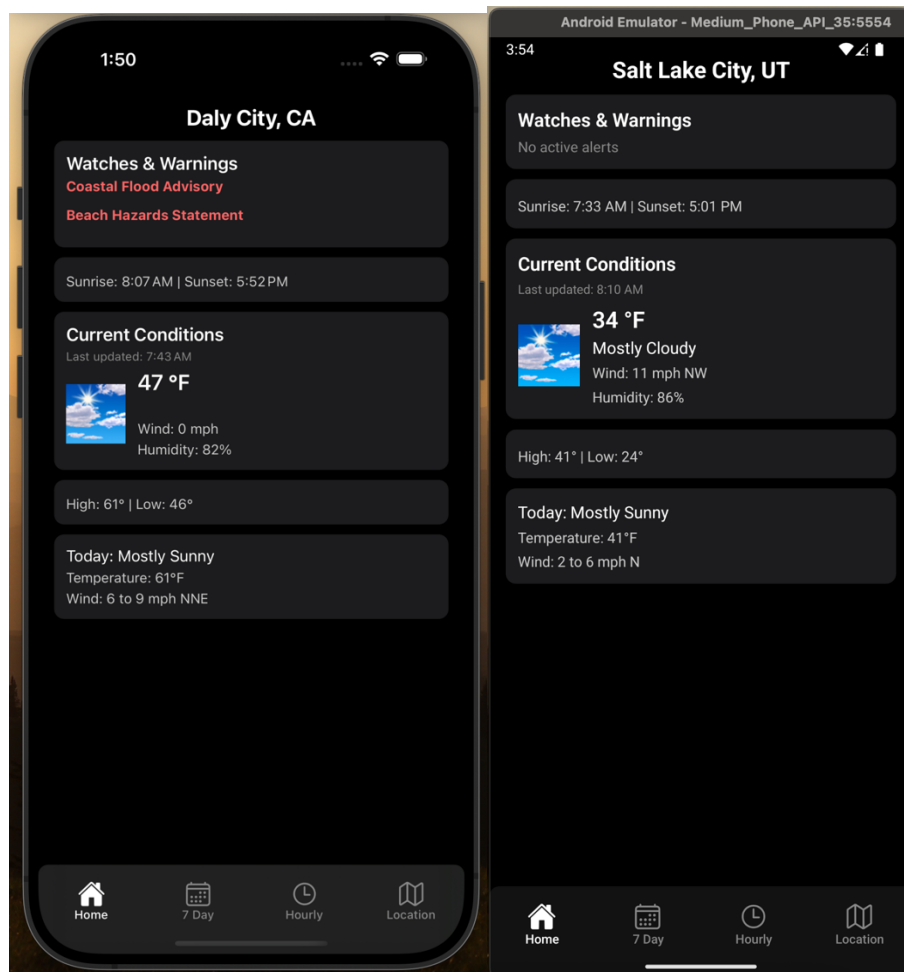


Figure 2: Index/Home screen on an iOS emulated device

The 7-day screen successfully loads the data for each 12-hour period in the next 7 days. This is separated into a scrollable view of cards, where each card shows the period, the high or low temperature (depending on day or night), and a detailed forecast description for the period. Each card also displays an icon for visual representation of the forecast. See below for a screenshot of the 7-day screen on both an emulated iOS and Android device.



Figure 3: 7-day screen on an iOS and Android emulated device

The hourly screen successfully loads the forecast for each hour in the available hourly forecast data. This could easily be trimmed to only the next 24, 48, 72 hours for simplicity, but it currently loads all periods. This screen is visually similar to the 7-day screen for consistency of

design and is a scrollable view of cards. Each card displays the period, short forecast description, temperature, humidity, wind, and precipitation chance. See below for a screenshot of the hourly forecast screen on both an emulated iOS and Android device.

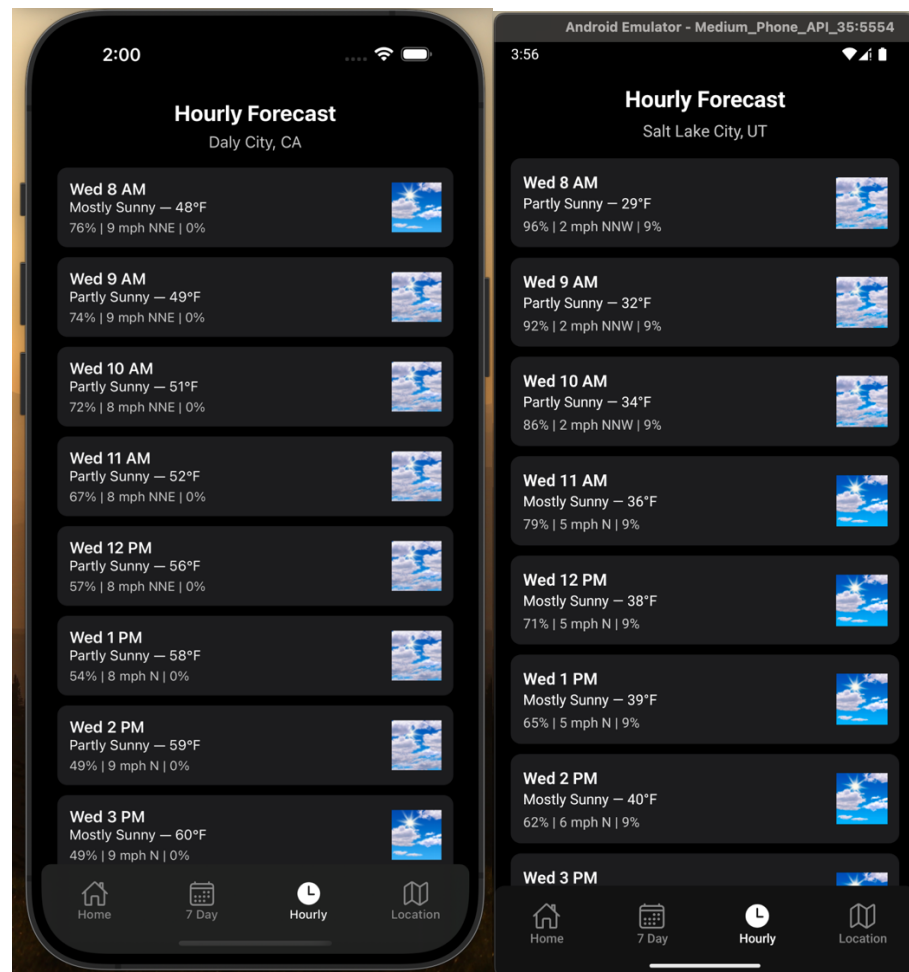


Figure 4: Hourly Screen on an iOS and Android emulated device

The location screen successfully displays 3 user input options to select a location. The user can search for a location via text input, select their current location via a button, or use the fully responsive and interactive map to navigate to any point for a location specific point forecast. The search bar offers auto completion using Geoapify API calls to make for a more user-friendly experience. React Native offers a very usable maps library to provide maps that are

fully functional in both Android and iOS, which makes this user experience seamless across devices. The map also clearly indicates the selected location via a place marker icon, and the search bar auto-populates the location if selected using a different method (i.e. via map click). See below for a screenshot of the location screen on both an emulated iOS and Android device.

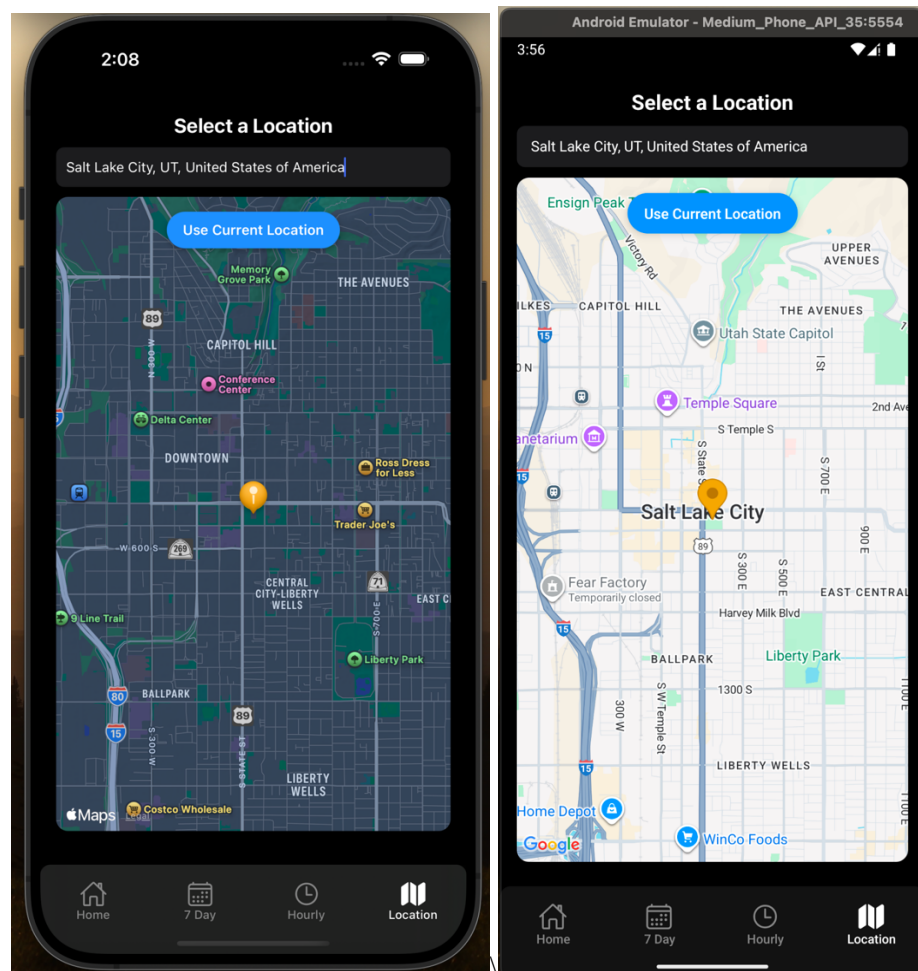


Figure 5: Location screen on an emulated iOS and Android device

Application-wide the software allows several other useful features. The home, 7-day, and hourly screens all allow pull to refresh which fetches the latest data by making fresh API calls. The application also offers offline use, which utilizes local caching to store the last fetched data and fall back to that data in the event of a network connection loss. Navigation between screens is handled by a simple navigation bar at the bottom of the screen. Each screen is labeled with a name and an icon, and the current screen is highlighted for clarity. The application has also been designed to work with both light and dark mode to match all user preferences, and it utilizes system settings to inherit that instead of relying on a separate app-based user selection.

Overall, the largest limitation of CAPE is the limited feature set. While the intent of the application is to remain simple, there are several other features that would add significant value to future releases/updates. These include the ability to save favorite locations, so users don't have to select a location manually, the addition of horizontal swiping for cross-screen navigation, and more fine-tuning of the user interface to assist with accessibility concerns and make the app as user friendly as possible. Of course, there is an endless list of features that can be added, but based on personal use and feedback from testers this is the best starting point. Time was the limiting factor in implementing these final features which are not currently available in the application.

Conclusion

In conclusion, React Native and the simple and understandable design of CAPE were the right choices. The application is fully functional and works great on both Android and iOS devices. As mentioned in the limitations, there are several features that can and should be added to make for an improved user experience, but these must be chosen very selectively. After the

conclusion of the course; the ability to save favorite locations, addition of swiping for cross-screen navigation, and more fine-tuning of the user interface to assist with accessibility concerns and make the app as user friendly as possible will be the main goals of the project. In its current implementation, CAPE successfully fills a market gap by offering a cross-platform mobile application for current weather and weather forecasts while utilizing the National Weather Service for accurate data. CAPE also integrates one of the most useful features, point forecasts, that are currently only available on the web (weather.gov) while expanding functionality to allow for offline use relying on caching. Several other small features, such as sunrise and sunset times and using current location make for a seamless and improved experience over any alternative offering. Overall, the project was a functionally success and can easily be added onto to continue refinement in the future.